

E4-TP3 : ÉCRAN D’AFFICHAGE

Prérequis :

- E4-Mémento1 : Pseudocode – Python – C/C++
- E2 : Algorithmique

Matériels et documentation :

- VS Code + PlatformIO
- NodeMCU ESP32
- Module OLED SSD1306

Durée :

4H00

Problématique

Comment afficher des images sur un écran OLED piloté par un ESP32 ?

Objectifs

À la fin de la séance, vous devrez être capable de :

- Instancier des objets en langage C++
- Appeler des méthodes en langage C++

On vous demande de mettre en œuvre un module d’affichage graphique et de réaliser des tests unitaires pour afficher du texte et des images afin de réaliser un écran d’affichage pour une baie de serveur informatique.

1 PRÉSENTATION DU MODULE D’AFFICHAGE

Le **module d’affichage** que vous allez mettre en œuvre est composé d’un **écran OLED 128x64** de 0,96 pouce et d’un **contrôleur SSD1306**.

Technologie de l’écran : OLED (*Organic Light-Emitting Diode*), contrairement aux écrans LCD traditionnels, les écrans OLED n’ont pas besoin de rétroéclairage. Chaque pixel produit sa propre lumière, ce qui permet d’avoir des noirs plus profonds et des couleurs plus vives.

Contrôleur : Il gère les opérations d’écriture et de contrôle sur l’écran, facilitant ainsi l’interaction avec l’écran pour les microcontrôleurs et les systèmes embarqués.

Connectique : Le module peut être connecté à un microcontrôleur ou à d’autres périphériques via une interface **I²C** (ou SPI).

Polyvalence : Le module peut afficher du texte, des chiffres, des formes géométriques simples et même des images. Il existe des fonctions de dessin et de texte fournies par les bibliothèques logicielles correspondantes.



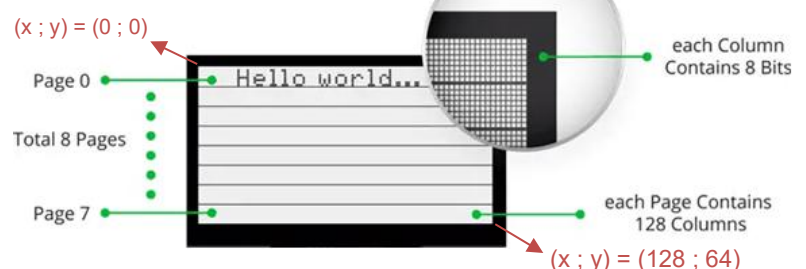
Photo 1 : Module d’affichage OLED 128x64 Jaune et bleu

2 ACTIVITÉ 1 : CÂBLAGE ET TEST DU MODULE D’AFFICHAGE

2.1 CARACTÉRISTIQUES PRINCIPALES

- (01) D’après la présentation du module d’affichage, expliquer pourquoi les écrans OLED consomment moins d’énergie que les écrans LED.
- (02) Sachant que 1 pixel est codé sur un bit, calculer la taille de la mémoire tampon (*buffer*) du contrôleur SSD1306 en kio. En déduire la taille en octets d’une page mémoire.

Figure 1 : Cordonnées, colonnes et pages mémoires de l’écran OLED



2.2 INTERFACE I²C

- (03) Relever l'adresse physique inscrite sur le module d'affichage (l'adresse est programmée en fonction de la présence de la résistance R11 ou R12). En déduire l'adresse I²C du module sachant qu'il faut supprimer de l'adresse physique le bits de poids faible (LSB).
- (04) Relever sur le brochage de la figure 2, les broches à connecter au module d'affichage. Compléter le tableau ci-dessous. Câbler le module d'affichage sur la carte nodeMCU ESP32 en utilisant les couleurs de fil proposées.



Figure 2 : Logo I²C

NodeMCU ESP32	Module d'affichage OLED	Couleur des fils
...	GND	Noir
3V3	VCC	Rouge
...	SCL (clock)	Blanc ou jaune
...	SDA (data)	Bleu ou vert

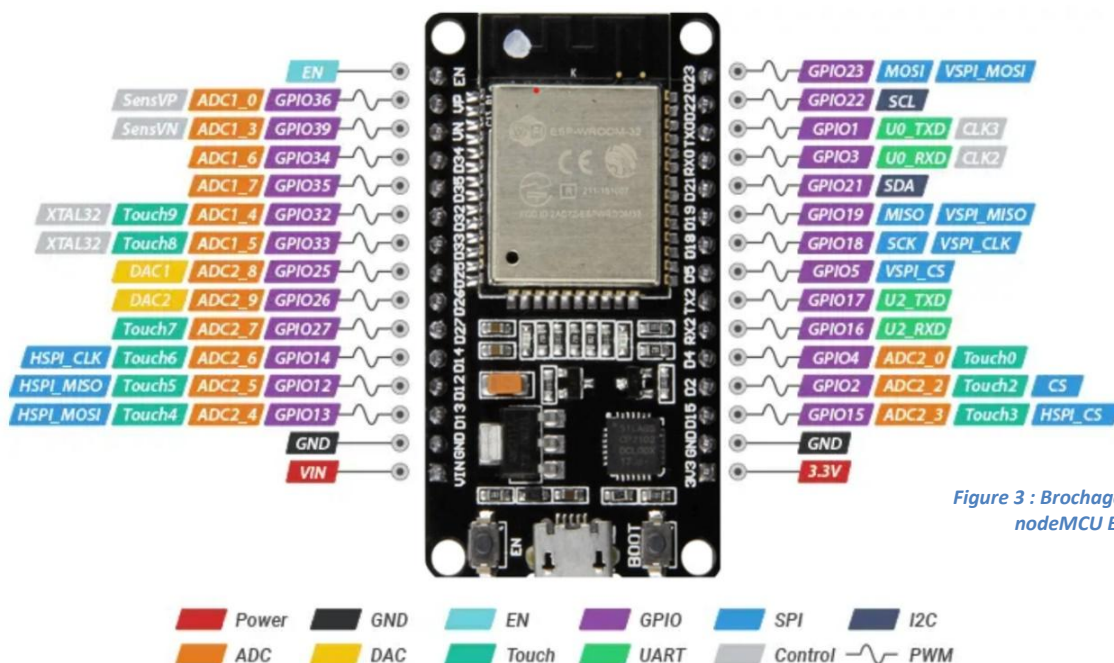


Figure 3 : Brochage de la carte nodeMCU ESP32

2.3 INTRODUCTION À LA PROGRAMMATION ORIENTÉE OBJET (POO)

Nous devons commander par programmation un écran OLED. Cet écran possède une résolution en pixel, une luminosité réglable et une adresse sur le bus I²C, il peut afficher du texte, des images, faire du défilement...

En **POO**, on pourrait créer une **classe** nommée **EcranOLED** qui aurait des **attributs** comme une résolution en **hauteur** et en **largeur**, une **adresseI2C** sur le bus I²C et des **méthodes** comme **demarrerEcran()**, **afficherImage(image)**, **afficherTexte(texte)**...

La classe **EcranOLED** pourra être utilisée pour représenter n'importe quel écran OLED dans notre programme, avec ses propres attributs et ses propres comportements. Par exemple, on peut créer un **objet** **ecran1** pour un écran de résolution 128x32 et d'adresse I²C 0x3C, et un **objet** **ecran2** pour un écran de résolution 128x64 et d'adresse I²C 0x3D.

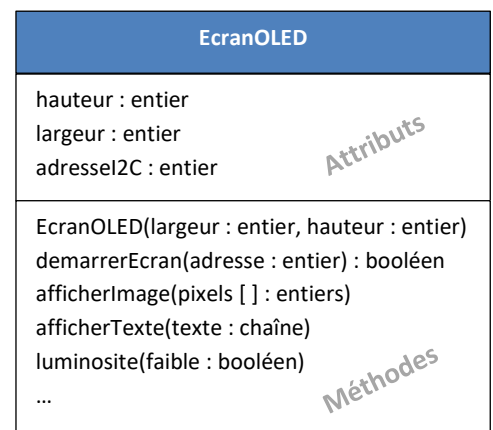


Figure 4 : Pictogramme UML de la classe EcranOLED

Et on peut utiliser les méthodes avec chaque objet pour simuler les actions qu'on peut faire avec chaque écran indépendamment, comme les démarrer, afficher une image, du texte...

Syntaxe C++

```

EcranOLED ecran1 (128, 32);
EcranOLED ecran2 (128, 64);

ecran1.demarrerEcran(0x3C);
ecran1.afficherTexte("Hello");

byte image1 [] = {...};

ecran2.demarrerEcran(0x3D);
ecran2.afficherImage(image1);
    
```

Syntaxe de la construction d'un objet :

`Nom_classe nom_objet (...)`

Appel d'un constructeur particulier
 Constructeur = méthode ayant le même nom que la classe
 Aucune parenthèse si utilisation du constructeur par défaut

Syntaxe de l'utilisation d'un objet avec une méthode :

`nom_objet.nom_méthode( )`

2.4 TEST UNITAIRE DU FONCTIONNEMENT DE L'ÉCRAN

- (05) Créer un nouveau projet nommé **projet_3_esp32_OLED**. Importer dans votre projet la bibliothèque **Adafruit SSD1306** contenant la classe permettant de créer et de piloter des écrans OLED avec contrôleur SSD1306. Compléter le fichier **main.cpp** avec les directives préprocesseurs pour :

- Inclure le fichier **Adafruit_SSD1306.h**
- Créer les symboles **SCREEN_WIDTH**, **SCREEN_HEIGHT** et **SCREEN_ADDRESS** avec leur valeur associée.

Vous utiliserez pour les questions suivantes la documentation de la classe `Adafruit_SSD1306` :

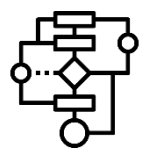
https://adafruit.github.io/Adafruit_SSD1306/html/index.html

- (06) Déterminer le constructeur à utiliser dans notre projet. Compléter le fichier **main.cpp** avec l'instruction pour construire un objet appelé **screen1** (en utilisant les valeurs par défaut du constructeur). Cet objet aura une portée globale dans le code.
- (07) Écrire le code dans la fonction **setup()** pour réaliser l'algorithme suivant :

```

SI l'objet screen1 n'est pas initialisé ALORS          // retour de la méthode begin()
    Afficher sur le port série « OLED screen : ERROR »
    Ne rien faire                                     // boucle infinie qui ne fait rien
FIN_SI

Afficher sur le port série « OLED screen : OK »
// Vider la mémoire tampon
Effacer l'écran                                     // méthode clearDisplay()
// Remplir la mémoire tampon
Placer le curseur à la position x=0 et y=0           // méthode setCursor()
Choisir la couleur blanche pour le texte             // méthode setTextColor()
Choisir la taille 2 pour le texte                   // méthode setTextSize()
Écrire sur l'écran « Hello » avec un retour à la ligne // méthode println()
Choisir la taille 3 pour le texte
Écrire sur l'écran « Hello » avec un retour à la ligne
Choisir la taille 1 pour le texte
Écrire sur l'écran « Hello » avec un retour à la ligne
// Transférer la mémoire tampon sur l'écran
Afficher sur l'écran le contenu du buffer            // méthode display()
    
```



Algorithme de la fonction **setup**

3 ACTIVITÉ 2 : AFFICHAGE D'IMAGES

3.1 ÉCRAN DE BIENVENUE

Nous souhaitons dans cette première partie de l'activité concevoir un écran d'accueil au démarrage de la carte nodeMCU ESP32.

(08) Vous modifierez le code que vous compilerez et téléverserez dans la carte pour obtenir le fonctionnement suivant (codage de la fonction setup) :

- Afficher au centre dans la partie jaune de l'écran OLED le message « Bienvenue »,
- Afficher au centre de la partie bleue de l'écran OLED, le logo de la section CIEL-IR pendant 1 seconde, puis le faire défiler, attention le texte restera fixe.

Vous trouverez ci-dessous le code au format Arduino de l'image de la section.

```
// 'logo ciel-ir', 69x32 px
const byte logo_cielir [] = {
  0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf8, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
  0xff, 0xf8, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf8, 0xff, 0xc0, 0x00, 0x00, 0x00,
  0x00, 0x00, 0x1f, 0xf8, 0xff, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x07, 0xf8, 0xfc, 0x00, 0x00,
  0x00, 0x00, 0x00, 0x00, 0x03, 0xf8, 0xf8, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0xf8, 0xf8,
  0x07, 0xff, 0xff, 0xff, 0xe0, 0x00, 0x00, 0xf8, 0xf0, 0x3f, 0xff, 0xff, 0xff, 0xe0, 0x00, 0x00,
  0xf8, 0xf0, 0x7f, 0xff, 0xff, 0xff, 0xe0, 0x00, 0x00, 0x78, 0xf0, 0xff, 0xfc, 0xff, 0xf1, 0xe0,
  0x00, 0x00, 0x78, 0xf0, 0xfc, 0x08, 0xff, 0xf1, 0xe1, 0x8f, 0xc0, 0x78, 0xf0, 0xf8, 0x08, 0xff,
  0xf1, 0xe3, 0x8f, 0xe0, 0x78, 0xf0, 0xf0, 0xf0, 0xf1, 0xe3, 0x8e, 0xf0, 0x78, 0xf0, 0xe1,
  0xef, 0xf9, 0xf1, 0xe3, 0x8e, 0x70, 0x78, 0xf0, 0xe3, 0xf8, 0xe0, 0x71, 0xe3, 0x8e, 0x70, 0x78,
  0xf0, 0xe3, 0xf8, 0xc2, 0x31, 0xe3, 0x8f, 0xf0, 0x78, 0xf0, 0xe3, 0xf8, 0xc6, 0x31, 0xe3, 0x8f,
  0xe0, 0x78, 0xf0, 0xe1, 0xf8, 0xc0, 0x31, 0xe3, 0x8f, 0xc0, 0x78, 0xf0, 0xe0, 0xc8, 0xc0, 0x31,
  0xe3, 0x8e, 0xe0, 0x78, 0xf0, 0xf0, 0x08, 0xc6, 0x31, 0xe3, 0x8e, 0xf0, 0x78, 0xf0, 0xf8, 0x08,
  0xe0, 0x31, 0xe3, 0x8e, 0x70, 0x78, 0xf0, 0xfc, 0x18, 0xf0, 0xf1, 0xe1, 0x8c, 0x38, 0x78, 0xf0,
  0x7f, 0xff, 0xff, 0xff, 0xe0, 0x00, 0x00, 0x78, 0xf0, 0x3f, 0xff, 0xff, 0xff, 0xe0, 0x00, 0x00,
  0xf8, 0xf8, 0x1f, 0xff, 0xff, 0xff, 0xe0, 0x00, 0x00, 0xf8, 0xfc, 0x00, 0x00, 0x00, 0x00, 0x00,
  0x00, 0x01, 0xf8, 0xfe, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x03, 0xf8, 0xff, 0x00, 0x00, 0x00,
  0x00, 0x00, 0x00, 0x07, 0xf8, 0xff, 0xe0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x3f, 0xf8, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xff, 0xf8, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf8
};
```



3.2 AFFICHAGE D'INFORMATIONS

Dans cette partie, nous compléterons le code précédent pour afficher des informations sur la température et le taux d'humidité d'une baie de serveur.

(09) Vous modifierez le code que vous compilerez et téléverserez dans la carte pour répondre aux demandes suivantes (codage de la fonction loop) :

- Afficher au centre dans la partie jaune de l'écran OLED le message « Baie 1 »,
- Afficher dans la partie bleue de l'écran OLED, deux icônes représentant respectivement une température et un taux d'humidité,
- Afficher des valeurs aléatoires de température et d'humidité à côté des icônes toutes les minutes, ne pas oublier de placer les unités (°C et %).



Outil de conversion d'image au format Arduino : <https://javi.github.io/image2cpp/>